

SERVERLESS CONTROL PLANE, CONTAINER DATA PLANE A GOVERNANCE BOUNDARY FOR MULTI-TENANT AI PLATFORMS

SHAYAN GHASEMNEZHAD, GIACINTO PAOLO (GP) SAGGESE, PAUL SMITH, HEANH SOK, AND
VEDANSHUBHAI JOSHI

ABSTRACT. Multi-tenant AI platforms fail in a plain, dangerous way. Services learn to make their own guesses about identity, tenancy, and capability. A query service trusts a caller-supplied tenant hint. A job runner accepts a schema prefix from a request payload. An OpenAPI document advertises endpoints for modules a tenant never bought. None of these decisions looks dramatic in isolation. Together, they turn governance into folklore.

We describe a production architecture that puts the folklore behind a boundary. A serverless control plane materializes identity, effective tenant context, module capability, API surface, impersonation state, and tenant-scoping signals. A container data plane executes workloads in Kubernetes, Lambda-backed modules, and on-demand Fargate jobs without owning policy. The important property is not AWS Lambda or EKS by themselves; it is the split. The control plane decides and attests. The data plane executes inside injected scope.

We report eight months of operational experience at Causify AI. The system concentrates shared governance in a 135-line Lambda middleware spine, unifies enforcement across 12 data-plane services, reduces visible API surface by 67–90% for limited-module tenants from a 48-endpoint base, and moves five of six common tenant lifecycle operations to reviewed data changes instead of deployments. From 1,368 production observations, full `/v2/me` materialization at 512 MB shows 856 ms P50 latency; ETag revalidation handles 91% of steady-state traffic at 68 ms P50. Structured-log correlation found zero actor/effective-tenant mismatches within the limits of the detection method. The pattern has been useful in production: make tenancy explicit, inject scope downstream, and stop asking every service to rediscover the truth.

Keywords. multi-tenant SaaS; AI platform; authorization; serverless architecture; control plane; data plane; scope injection; confused deputy; API governance; AWS Lambda; Kubernetes

CONTENTS

1. Introduction	2
2. Governance Failures in Multi-Tenant Platforms	2
3. Architecture Overview	3
4. Control Plane Design	5
5. Container Data Plane Integration	6
6. Security Model	9
7. Operations and Reliability	10
8. Evaluation	11
9. Related Work	13
10. Limitations and Future Work	13
Practitioner Takeaways	14
11. Conclusion	14
References	14
About the Authors	15

Date: July 2026.

1. INTRODUCTION

A platform is a set of promises. This tenant’s data stays with this tenant. This actor may perform this action. This module exists for this role. This job is running under the scope it claims to carry.

Small systems keep those promises with local code and human memory. A handler checks a role. A service reads a header. A worker receives a schema name. A support engineer knows which tenant should be visible. Then the product grows. Tenants multiply. Modules become products. Support workflows need impersonation. Batch jobs and long-running services start sharing the same data model. The old local checks still work, right up until they do not.

The failure is usually not cinematic. It is boring and therefore hard to notice. One service trusts a tenant field supplied by its caller. Another re-derives tenancy from stale membership data. A generated API spec includes routes for modules the effective tenant cannot use. A job launches with the right identity but the wrong database prefix. The incident report never says “the architecture lacked taste,” but that is often the root cause.

This paper describes the architecture we use to keep those decisions in one place. The pattern has two halves:

- A *serverless control plane* resolves identity, effective tenant context, module capability, API surface, impersonation state, and scoping signals.
- A *container data plane* runs product services, analytics jobs, data workflows, and on-demand compute using the scope injected by the control plane.

The control plane is intentionally thin. It is not a place to put business logic, model execution, or every convenient helper function. It is the place where the platform answers the questions other services should not be answering for themselves: who is this actor, which tenant context is effective, which modules exist, which API surface should be visible, and which scope must flow downstream.

The data plane is the execution fabric. In our deployment it includes EKS services, module Lambdas, Airflow-driven workflows, and Fargate jobs. The important rule is simple. The data plane can receive scope. It does not own the authority to derive it.

Contributions. This paper makes four contributions:

- It names three governance failure modes in multi-tenant platforms: scope ambiguity, governance scatter, and surface drift.
- It describes a concrete control-plane pattern built around a versioned identity contract, server-side scope injection, module-gated routing, OpenAPI surface filtering, and auditable `view_as`.
- It shows how the pattern integrates with a container data plane running EKS services, DataMap pipeline orchestration, Optima-style optimization workloads, Airflow DAGs, module Lambdas, RDS Proxy, PostgreSQL schema isolation, and Fargate jobs.
- It reports production measurements from Causify AI: 12 governed data-plane services, 135 lines of shared Lambda governance, 67–90% API-surface reduction for limited-module tenants, 91% identity-contract cache hits, and zero actor/effective-tenant mismatches detected across eight months within the limits of the audit method.

The pattern is not a new authorization theory. It is a way to make existing authorization ideas operationally hard to ignore. In a multi-tenant platform, governance should feel dull. Dull is good. Dull means the same thing happens on Tuesday afternoon as happened during the sales demo.

2. GOVERNANCE FAILURES IN MULTI-TENANT PLATFORMS

Multi-tenant platforms usually start with a clean story. A tenant owns data. Users belong to tenants. Roles grant capabilities. Services enforce access. The model is fine. The implementation is where it frays.

2.1. Scope ambiguity. A service receives a tenant identifier from a caller and treats it as truth. A request header says `tenant=acme`; a query parameter says `schema=demo_acme`; a job payload carries an old prefix. Each value is plausible. None is authoritative unless it was derived from authenticated context by the system that owns tenancy.

That is the confused-deputy shape in platform clothing [15]. A downstream service acts with more authority than the caller should have, or under a tenant the actor did not intend, because the service trusted a scope hint it should have treated as untrusted input.

2.2. Governance scatter. The second failure is duplication. Every service that joins the platform implements its own version of identity resolution, module gating, role checks, impersonation rules, and audit logging. One service validates JWTs against Cognito. Another trusts an internal caller. Another reads a tenant table directly. Another copies old middleware and never gets the patch.

At 12 services, a single governance rule change becomes 12 separate edits, 12 review paths, and 12 chances to drift. The code volume is not the main problem. The problem is that the platform no longer has one answer.

2.3. Surface drift. The third failure is what the platform advertises. API documentation and generated clients often expose every endpoint the product knows how to serve, not every endpoint a given tenant may use. Clients discover disabled modules, call their routes, and receive errors that look like product bugs. Operators see support tickets. Security teams see unnecessary attack surface.

The visible API surface should be a consequence of effective tenant capability. If a tenant has not licensed a module, the API spec should not invite them to call it.

2.4. Design goals. The two-plane boundary is designed around five goals.

G1 Explicit tenancy.:

Actor identity and effective tenant context are distinct fields. Support impersonation changes the effective context, never the actor.

G2 Scope injection.:

Scope flows downstream from the control plane. It does not flow upstream from callers.

G3 Minimized surface.:

Tenants see only the routes and modules they are allowed to use.

G4 Auditable impersonation.:

Operators can view as a tenant without shared credentials or unattributed actions.

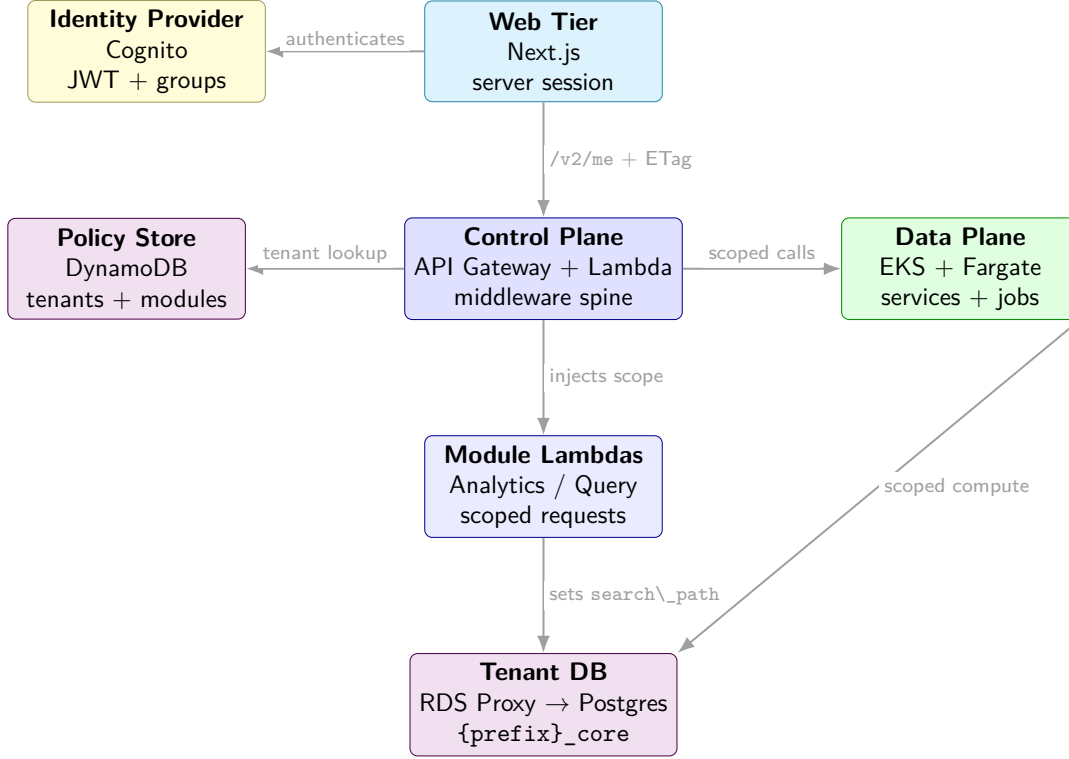
G5 Governance as data.:

Routine tenant operations become reviewed configuration changes. New module types still require code.

These goals are modest on purpose. The point is not to build a universal policy engine. The point is to remove ambiguity from the high-frequency paths that make SaaS platforms painful to operate.

3. ARCHITECTURE OVERVIEW

Figure 1 shows the boundary. The control plane combines API Gateway, Lambda, DynamoDB, Cognito validation, and a small middleware spine. It owns the decision path. The data plane combines EKS services, module Lambdas, Fargate jobs, Airflow-driven workflows, and tenant-scoped storage. It owns execution.



Boundary rule. Tenancy and capability decisions originate in the control plane. Execution systems receive explicit scope and run workloads. They do not rediscover policy.

FIGURE 1. The control-plane/data-plane boundary. The control plane resolves identity, effective tenant context, modules, impersonation, API surface, and downstream scope. The data plane executes within that injected scope.

3.1. The `/v2/me` contract. The control plane exposes a versioned identity contract that the web tier calls before page rendering. It is similar in spirit to an OpenID Connect UserInfo endpoint [20], but it carries platform-specific tenant and module state.

A simplified response is shown below.

```

1 {
2   "actor": {
3     "email": "ada@acme.example",
4     "tenant_id": "platform-a",
5     "user_role": "admin"
6   },
7   "view_as": {
8     "tenant_id": "acme",
9     "user_role": "member",
10    "schema_prefix": "acme"
11  },
12  "impersonatable": [
13    { "tenant_id": "acme", "roles": ["demo", "member", "admin"] }
14  ]
15 }

```

LISTING 1. Simplified identity and effective-context contract.

The important field is `schema_prefix`. It appears under `view_as` because database scope follows the effective tenant, not the support engineer’s home tenant. When no impersonation is active, `view_as` simply mirrors the actor’s own tenant context.

The endpoint uses ETag and If-None-Match semantics from RFC 9110 [13]. On a cache hit, the Lambda returns 304 Not Modified with no body. The fingerprint covers identity and role fields; module changes are operator actions and take effect at the next full materialization. That trade-off gives the platform cheap, frequent revalidation without pretending policy changes are end-user events.

3.2. What belongs on each side. The boundary is practical, not theological. Authentication, tenant lookup, module gating, impersonation validation, API-surface filtering, and scope derivation live in the control plane. Query execution, model execution, workflow orchestration, data transformation, exports, and customer-facing computations live in the data plane.

The architecture becomes easier to reason about once that split is enforced. When a data-plane service receives a request, it should not ask, “which tenant am I really serving?” It should already know. The control plane has answered that question upstream.

4. CONTROL PLANE DESIGN

The control plane is a Python Lambda behind API Gateway. Its job is deliberately narrow: authenticate, derive context, gate modules, minimize surface, and log enough to make incidents explainable. The implementation is small enough to review, but important enough to treat like production security infrastructure.

4.1. Middleware spine. Every handler passes through the same middleware chain. In the deployed system, this is implemented with AWS Lambda Powertools decorators [10]. The spine has three layers.

- (1) **Authentication and context.** The middleware validates Cognito JWTs [2], resolves tenant membership from the platform table instead of trusting token claims alone, derives role and `schema_prefix`, and builds the set of available impersonations.
- (2) **Response hardening.** Standard security headers are set at the boundary: HSTS, Content-Security-Policy, X-Frame-Options, and X-Content-Type-Options.
- (3) **Conditional revalidation.** The middleware compares the response fingerprint with If-None-Match and returns 304 Not Modified when the identity contract has not changed.

This is not glamorous code. That is the point. Shared governance should be reviewable and dull.

4.2. Scope injection. Tenant scope is derived from the tenant record and injected into downstream calls. It is never accepted from browser input, query parameters, or ad hoc service headers.

For query and analytics modules, the control plane passes `schema_prefix` as a required field in the internal request payload. Downstream Lambdas reject missing or empty scope with a bad-request error. They re-validate syntax before using the value. They do not independently decide whether a syntactically valid prefix belongs to the caller; that authority remains in the control plane.

This distinction matters. Downstream checks catch malformed or absent scope. They cannot catch a wrong-but-well-formed scope derived by the control plane. The architecture therefore makes the control plane the load-bearing derivation point and makes the residual risk explicit.

4.3. Module gating and surface minimization. Each route carries a module requirement. In the Python services this is expressed as a decorator such as `@requires_module("grid")`. Before a handler body executes, middleware compares the required module with the enabled-module list attached to the effective tenant context. Missing module, no handler execution.

The same enabled-module set filters the generated OpenAPI document. The `/v1/openapi.json` endpoint removes unlicensed paths and prunes unreferenced schema definitions before serving the

spec. That avoids a common platform smell: endpoints that are officially documented, operationally denied, and still somehow treated as customer-facing promises.

Filtering happens after impersonation resolution. A support engineer viewing as `acme/member` sees the API surface for `acme/member`, not the support engineer’s broader internal surface.

4.4. Auditable view_as. Support engineers sometimes need to see what a customer sees. Shared admin accounts are the tempting shortcut. They are also audit poison. They erase actor attribution, concentrate risk into a single credential, and make every subsequent log line weaker.

The `view_as` mechanism keeps actor and effective context separate. The actor field remains the authenticated support engineer. The effective context changes to the selected tenant and role. Every audited action can carry both identities.

The web tier signs the `X-View-As` payload with HMAC-SHA256. The signed string includes tenant, role, and a per-activation nonce. The signing key is held server-side. The browser never receives it. The nonce is registered in DynamoDB at activation time, and logout or tenant switching revokes it. If nonce registration fails, impersonation fails closed.

A simplified sketch follows. Internal variable names are intentionally omitted.

```

1 def activate_view_as(actor, tenant_id, role):
2     allowlist = get_available_impersonations(actor)
3     if (tenant_id, role) not in allowlist:
4         raise Forbidden()
5     nonce = create_nonce()
6     payload = f"tenant_id={tenant_id};role={role};nonce={nonce}"
7     signature = hmac_sha256(server_side_key(), payload)
8     register_nonce(actor, nonce)
9     store_in_encrypted_session(payload, signature)
10    return ok()

```

LISTING 2. Simplified view-as signing flow.

The details are mundane, but the guarantee is valuable: concurrent support sessions remain independently scoped and attributable without shared credentials.

5. CONTAINER DATA PLANE INTEGRATION

The data plane runs the work that would make a control plane miserable: APIs, long-running services, data pipelines, optimization workloads, scheduled jobs, and integration workers. In the deployed system, that means EKS namespaces, Horizon/Grid-style module Lambdas, DataMap pipeline orchestration, Optima-style optimization services, Airflow DAGs, connector jobs, and Fargate tasks.

This is why the paper uses “container data plane” rather than “Kubernetes backend.” The execution fabric is mixed on purpose. Some work belongs in services, some in Lambdas, some in short-lived containers, some in scheduled pipelines, and some in connector workers. The governance rule should not change with the substrate.

5.1. Representative execution surfaces. Table 1 summarizes the main surfaces governed by the control plane. The details differ; the contract does not. Each surface receives a small scope envelope, rejects missing scope, and avoids rediscovering tenant identity from user input.

Surface	Workload shape	Control-plane responsibility
Horizon/Grid module Lambdas	Tenant-scoped query and analytics APIs	Gate module routes, inject <code>schema_prefix</code> , preserve actor/effective context
DataMap orchestration	Subject $\rightarrow$ Analysis $\rightarrow$ Graph pipelines and graph-version history	Attach tenant, user, subject, analysis, and version scope before pipeline execution
Optima-style optimization	Long-running portfolio and scenario optimization jobs	Launch scoped compute and keep job intent tied to effective tenant context
Airflow DAGs	Scheduled and ad-hoc data workflows	Receive scope and environment context rather than resolving membership inside DAG code
Connector workers	External-system sync, webhook, and ingestion jobs	Bind connector metadata, credentials, and job events to tenant-owned records

TABLE 1. Representative data-plane surfaces. The substrate varies, but governance enters through the same control-plane boundary.

5.2. Module Lambdas: Horizon and Grid. Horizon and Grid are useful examples because they look like ordinary product APIs until tenant scope gets involved. Horizon serves module-specific queries and derived views. Grid serves time-series analytics, predictions, and derived signals. Both sit behind module-gated routes in the control plane.

The control plane first resolves the actor and effective tenant, checks that the module is enabled, then delegates to the module Lambda with an explicit scope envelope. The downstream Lambda treats `schema_prefix` as required input. If it is absent, the request fails before any domain logic runs.

```

1 class TenantScopedHandler:
2     def __init__(self, query_params):
3         self.schema_prefix = query_params.get("schema_prefix")
4         if not self.schema_prefix:
5             raise BadRequestError("schema_prefix is required")
6         validate_schema_prefix(self.schema_prefix)
7         self.core_schema = f"{self.schema_prefix}_core"

```

LISTING 3. Simplified module Lambda scope check.

The Lambda does not decide whether `acme` is the right tenant. It only checks that the scope is present and syntactically safe. The authority for deriving `acme` remains in the control plane. That division is subtle but important. If every module Lambda is allowed to rediscover tenant membership, the boundary is gone.

5.3. Scope envelope. The envelope passed downstream is intentionally small. It carries just enough information for execution to be correct and auditable, not enough to turn every service into a policy interpreter.

Field	Typical consumers	Purpose
<code>actor_id</code>	Audit logs, support workflows	Preserves the real authenticated identity
<code>effective_tenant</code>	All tenant-scoped services	Names the tenant context in effect after optional <code>view_as</code> resolution
<code>effective_role</code>	Module APIs, UI contracts	Carries the role used for capability checks
<code>schema_prefix</code>	Horizon/Grid, query services, jobs	Selects the tenant database namespace, e.g., <code>{prefix}_core</code>
<code>modules</code>	API filtering, route guards	Declares enabled product modules for the effective context
<code>subject_id</code> , <code>analysis_id</code> , <code>version_id</code>	DataMap	Locates a stable artifact hierarchy for graph refinement and history
<code>connector_id</code> , <code>job_id</code>	Connector workers	Binds external-system sync to tenant-owned meta-data and credentials

TABLE 2. The scope envelope. The control plane derives it; data-plane systems consume it.

The point is not to standardize these exact field names forever. The point is to stop hiding tenancy in query parameters, environment variables, README files, and tribal memory.

5.4. DataMap as a scoped artifact pipeline. DataMap has a different shape from Horizon and Grid. It is closer to an interactive modeling studio: a user starts from a subject, creates analyses, generates or refines causal graphs, and keeps history across versions. The workflow is naturally iterative. A graph is not overwritten casually; it is refined, versioned, and attached to the subject and analysis that produced it.

Figure 2 shows the artifact hierarchy. The control plane does not need to understand graph semantics. It does need to attach the right tenant and user context before the pipeline creates, reads, or updates artifacts.

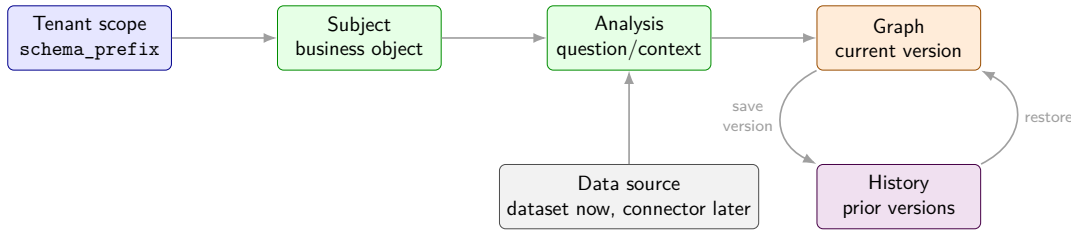


FIGURE 2. DataMap artifact flow. Tenant scope is attached before the pipeline touches subjects, analyses, graph versions, or connector-backed inputs.

A typical storage layout mirrors the same hierarchy. The exact table and bucket names are deployment details, but the key pattern is stable.

```

1 subject_key = f"tenant#{tenant_id}#subject#{subject_id}"
2 analysis_key = f"{subject_key}#analysis#{analysis_id}"
3 graph_key    = f"{analysis_key}#graph#{graph_id}#version#{version}"
4 object_key = (
5     f"tenants/{tenant_id}/subjects/{subject_id}/"
6     f"analyses/{analysis_id}/graphs/{graph_id}/{version}.json"
7 )

```

LISTING 4. Representative DataMap artifact key pattern.

That shape is deliberately boring. It makes version history auditable, lets multiple analyses exist under one subject, and leaves room for data sources to evolve from uploaded datasets to connector-backed queries without changing the tenant boundary.

5.5. Connector ingestion. Connector ingestion adds one more wrinkle: external systems. A connector has tenant-owned metadata, tenant-owned credentials, webhooks or polling jobs, and downstream events. The control plane owns the connector record and launches or receives work under tenant scope. Connector workers execute sync logic; they do not decide which tenant owns a token or destination.

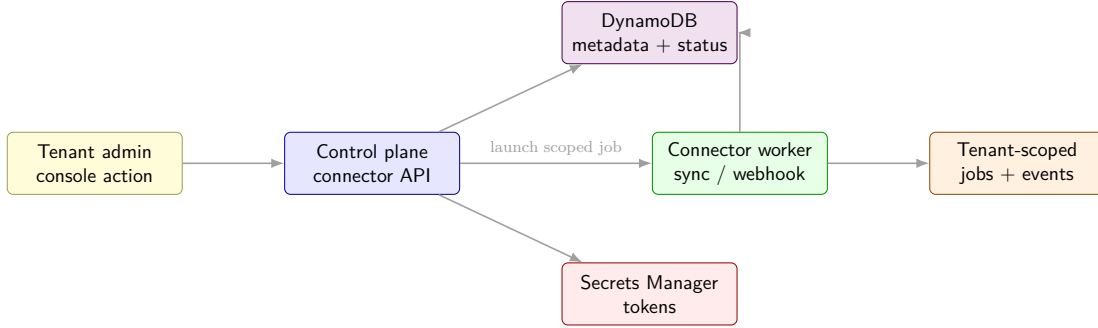


FIGURE 3. Connector ingestion boundary. The control plane owns tenant metadata and credential binding; workers perform sync and event handling inside the injected tenant scope.

This is the same pattern again. Interactive APIs, modeling pipelines, optimization jobs, and connector workers all receive scope. None of them should need a private theory of tenancy.

5.6. Database scoping. Database credentials are managed by AWS Secrets Manager [11] and reached through RDS Proxy [6]. Per-tenant isolation uses PostgreSQL schemas: module Lambdas set `search_path` to a tenant-specific schema such as `acme_core` before executing queries [22].

PostgreSQL does not allow parameter binding for `SET search_path`, so `schema_prefix` is validated before use. The control plane validates before forwarding. Module Lambdas re-validate before issuing the statement. This second check is defense in depth. It protects against malformed strings. It does not replace authoritative derivation.

5.7. On-demand jobs. Fargate jobs receive scope at launch time through the ECS `RunTask` payload [4]. The job does not perform a new user lookup and does not resolve tenant membership. It receives a scoped job intent and runs.

That rule keeps batch work inside the same governance model as interactive APIs. Jobs are not a back door. They are data-plane executions with a larger time budget.

5.8. One-directional dependency. The direction of dependency is deliberate. The control plane calls into the data plane. The data plane does not call back to ask who it is allowed to be. This avoids a circular governance path where every service is both execution engine and policy interpreter.

The resulting contract is simple enough for engineers to remember: if a service needs tenant scope, it receives it from the control plane. If a service needs to invent tenant scope, the design has already failed.

6. SECURITY MODEL

The architecture defends against four attack and failure classes: forged tenancy context, confused-deputy scope substitution, surface drift that exposes disabled endpoints, and unattributed operator action during impersonation.

6.1. **Layered controls.** The deployed system uses five layers.

- (1) **JWT validation.** A valid Cognito JWT is required, but token claims alone do not decide tenant scope. The middleware re-derives membership from the platform table.
- (2) **View-as guardrails.** Impersonation requires a server-side allow-list match, a signed payload, a live session, and a registered nonce.
- (3) **Scope injection.** Downstream services receive `schema_prefix` from the control plane and reject calls that omit it.
- (4) **Surface minimization.** Unlicensed routes are missing from the served API spec and return 403 if called directly.
- (5) **Network and transport hardening.** Data stores and compute services are not exposed as public shortcuts around the control plane.

No one layer is treated as magic. A valid JWT does not grant arbitrary scope. A visible UI control does not authorize a backend call. A syntactically valid schema prefix does not prove tenant membership. Each layer narrows a different failure mode.

6.2. **What remains risky.** The policy store becomes a critical surface. Moving policy from code to data is useful, but it changes the blast radius of a bad write. A faulty tenant record can break module access and scope for that tenant. A malicious write could do worse.

The mitigation is operational discipline: scoped IAM writes, reviewable changes, strict parsing, schema validation, audit logs, and a habit of treating policy updates with the same seriousness as code deploys. Configuration is not casual just because it is stored in DynamoDB.

The second residual risk is wrong derivation inside the control plane. If the control plane derives the wrong tenant scope and all logged fields agree with that wrong value, downstream services cannot detect the mistake. Section 8 discusses the detection gap honestly.

7. OPERATIONS AND RELIABILITY

The control-plane boundary helped most in the ordinary work of operating tenants. It made changes smaller, logs clearer, and failure modes less mysterious.

7.1. **Lifecycle operations.** Most tenant lifecycle changes are data changes:

- add a tenant,
- enable or disable a module,
- change module layout,
- grant or revoke an impersonation target,
- change a tenant schema prefix.

These updates touch the tenant policy store and are visible at the next full materialization. Adding a new module type still requires code: routes, handlers, decorators, defaults, tests, and documentation. That is the right split. Routine tenant operations should not redeploy 12 services. New product surface should still go through engineering review.

7.2. **ETag revalidation.** The `/v2/me` contract is called often enough that performance matters. Revalidating identity and effective context on every page load is valuable; re-sending the full payload each time is wasteful. ETag-based conditional requests keep the contract cheap enough to use as the platform’s source of truth.

The steady-state hit rate is 91%. A 304 Not Modified response returns at 68 ms P50. Full materialization is slower, but less frequent. That is a reasonable trade: correctness is checked often, while expensive identity assembly happens when it has something to say.

7.3. Logs and incident handling. Every request logs actor tenant, effective tenant, module context, and `schema_prefix` where applicable. The distinction between actor and effective tenant is not a compliance flourish; it is how support sessions remain explainable.

A representative incident query looks for mismatches between actor tenant, effective tenant, and impersonation state. That catches missing-prefix requests and non-impersonation context divergence. It does not catch a wrong value that was consistently derived upstream. The logs are useful, not omniscient.

7.4. Adoption path. Migration does not need a flag day. The practical sequence is:

- (1) deploy the control plane Lambda, tenant table, and `/v2/me` endpoint,
- (2) route selected service calls through the control plane,
- (3) replace local middleware with shared context handling,
- (4) add module decorators to routes,
- (5) move per-service module flags into the central tenant record,
- (6) turn on OpenAPI filtering for tenants with limited modules.

The migration of 12 services took approximately 160 engineer-hours across two sprints, including tests. That is not free. It is still cheaper than carrying a permanent governance tax in every service.

8. EVALUATION

The evaluation is a production case study from one organization, not a controlled experiment. The goal is to show what the boundary changed in practice and where the evidence is weaker than we would like.

8.1. Latency. Table 3 reports `/v2/me` full-materialization latency from 1,368 production observations at 512 MB Lambda memory, collected from CloudWatch metrics during March–April 2026. The full path includes JWT verification, membership lookup, tenant record read, impersonation-list assembly, and JSON serialization.

Metric	Latency (ms)
P50	856
P75	938
P90	1,038
P95	1,112
P99	1,176

TABLE 3. Full `/v2/me` materialization latency at 512 MB, based on 1,368 production observations from March–April 2026.

ETag-conditional revalidations dominate steady-state traffic at a 91% hit rate and return 304 `Not Modified` at 68 ms P50. At that hit rate, amortized per-page cost is approximately 139 ms. Cold starts add 300–600 ms in observed tail cases; Provisioned Concurrency is the obvious mitigation when a stricter latency budget is required [9].

8.2. Scope-injection checks. Scope injection is checked both statically and at runtime.

- CI verifies that route handler files apply module requirements and that Lambdas receiving `schema_prefix` reject missing or empty values.
- Runtime logs emit actor tenant, effective tenant, and `schema_prefix` per request. A CloudWatch Insights query surfaces non-impersonation mismatches for review.

Within this envelope, structured-log correlation detected zero actor/effective-tenant mismatches across eight months of production operation, from Q3 2025 through April 2026. Monthly impersonation-log audits and customer-reported issues were also clear.

The envelope matters. The method can catch missing scope and actor/effective mismatch. It cannot prove that the control plane never derived a wrong tenant value. If the wrong value is derived consistently, the logs will agree with themselves. We treat that as a limitation, not as an inconvenient detail to hide.

8.3. Surface filtering. The platform has 48 documented API endpoints in the measured surface: 5 core endpoints and 43 module-gated endpoints. Filtering the OpenAPI document by enabled modules reduces what limited tenants can discover.

Modules enabled	Endpoints visible	Reduction
0 (core only)	5	90%
1	16	67%
2	27	44%
3+	38	21%
Full access	48	0%

TABLE 4. Endpoint exposure after OpenAPI filtering. Synthetic module-tier profiles are derived from the 48-endpoint production surface.

Surface filtering does not replace backend authorization. It reduces discovery and support noise. Disabled endpoints are absent from the tenant’s spec and return 403 if called directly.

8.4. Governance concentration. Before the control plane, a governance policy change required deployment to each of 12 data-plane services, averaging roughly four engineer-hours per operation. After the migration, five of six common tenant lifecycle operations became DynamoDB record updates that take under five minutes. Only adding a new module type requires code.

The code concentration is also measurable. Shared JWT validation, role resolution, module gating, and scope derivation live in a 135-line Lambda middleware spine. Reimplementing comparable logic in 12 services would put the same risk in roughly 1,620 lines, before counting drift. The impersonation allow-list check is 67 lines in one place rather than 12 similar versions with slightly different behavior.

Table 5 summarizes the operational effects.

Question	Before	After
Tenant/module changes	Service deployments	Reviewed policy-store updates
Governance logic	Repeated across services	135-line shared middleware spine
API discovery	Broad generated spec	Filtered by effective modules
Impersonation	Risk of shared or ambiguous context	Actor/effective context logged separately
Migration cost	Ongoing per-service tax	160 engineer-hours across two sprints

TABLE 5. Operational effects observed after concentrating governance in the control plane.

8.5. When the pattern pays off. This pattern is probably not worth the ceremony for a three-service product with simple roles. It starts paying rent when the platform has multiple tenants, multiple modules, support impersonation, and enough services that nobody can confidently name every authorization path from memory. In our deployment, that line was crossed well before 12 services.

9. RELATED WORK

Authorization models. NIST SP 800-162 formalizes attribute-based access control [16]. OPA and Cedar provide richer policy languages for complex authorization logic [19, 12]. Zanzibar models relationship-based authorization at global scale [21]. The pattern in this paper does not replace these systems. It focuses on a narrower integration problem: deriving tenant context once, materializing it as a contract, and injecting scope into services that should not be interpreting identity for themselves. **AWS SaaS architecture.** The AWS SaaS Lens recommends control-plane/data-plane separation for multi-tenant SaaS systems [7]. Our contribution is a concrete instantiation of that advice for a production AI platform: a versioned identity contract, explicit actor/effective context, OpenAPI filtering, and schema-prefix injection into module and job execution paths.

OIDC and OAuth. The `/v2/me` endpoint resembles an OpenID Connect UserInfo endpoint [20]. The `view_as` mechanism resembles delegated context, but it is intentionally narrower than general OAuth delegation [14]. We did not need a universal identity framework. We needed a precise platform contract that support workflows and service calls could consume consistently.

Service mesh enforcement. Service meshes such as Istio and Linkerd can enforce network-level authorization [17, 18]. They are useful, but network authorization is not enough for module-level tenant policy, OpenAPI surface filtering, or PostgreSQL schema-prefix injection. We keep those decisions at the application boundary where tenant semantics are visible.

Serverless and container execution. Lambda, API Gateway, DynamoDB, EKS, and Fargate are standard cloud primitives [9, 1, 3, 5, 8]. The paper is not claiming novelty in the primitives. The useful part is the boundary between them: serverless functions for short governance decisions and containers for long-running execution.

10. LIMITATIONS AND FUTURE WORK

Single-organization evidence. All measurements come from one production deployment at Causify AI. The numbers are useful as operational evidence, not as a general benchmark.

AWS-specific implementation. The architecture uses API Gateway, Lambda, DynamoDB, Cognito, EKS, Fargate, RDS Proxy, and CloudWatch. Equivalent ideas exist on other clouds, but we have not validated the mapping.

Python-specific route enforcement. The deployed route decorator is Python-specific. Polyglot services need either a language-appropriate equivalent or enforcement moved closer to the gateway. **REST-first surface filtering.** OpenAPI filtering works well for REST APIs. GraphQL and gRPC need separate strategies: schema pruning, resolver-level capability checks, or gateway-level method filtering.

Control-plane derivation remains load-bearing. The strongest remaining risk is wrong derivation inside the control plane. Downstream services can reject missing or malformed scope, but they cannot independently verify a wrong tenant prefix that the control plane derived consistently. Future work includes signed scope envelopes, stronger policy-versioning fields, and automated tenant-matrix tests that exercise representative actor/effective combinations.

Policy authoring still needs better tools. Raw JSON policy records are easy to store and hard to love. Future tooling should include JSON Schema validation, diffs, previews, staged rollout, and CI gates for tenant/module changes. Policy-as-data should not become policy-by-vibes.

PRACTITIONER TAKEAWAYS

- **Materialize identity as a versioned contract.** Make actor, effective tenant, role, modules, impersonation, and scope visible in one response. Cache it, but treat caching as a correctness problem.
- **Derive scope once and inject it.** Let the control plane derive `schema_prefix` from authenticated tenant state. Downstream services should reject missing scope, not invent it.
- **Filter the API surface early.** If a tenant cannot use a module, the generated API spec should not advertise it. Backend authorization still enforces the final decision.
- **Keep impersonation attributable.** Support engineers need customer perspective. They do not need shared accounts or log lines that erase the actor.

11. CONCLUSION

The hardest bugs in multi-tenant platforms are often small enough to look harmless. A scope field comes from the wrong place. A generated spec shows too much. A worker accepts a tenant hint it should never have trusted. A support session loses track of the real actor. The platform still runs, but trust leaks out of it.

The control-plane/data-plane split described here is a way to stop that leak. The control plane materializes identity, effective context, module surface, impersonation state, and tenant scope. The data plane runs the work. That separation kept governance reviewable across 12 services, reduced endpoint exposure for limited tenants, moved routine tenant changes out of deployments, and made incident analysis less dependent on memory.

The thesis is intentionally plain: governance is part of the product. Put it behind a boundary, make it explicit, and keep it boring enough to operate. The data plane can be ambitious. The control plane should be calm.

REFERENCES

- [1] Amazon API Gateway. Amazon api gateway documentation. Documentation. Accessed 2026-07-15. URL: <https://docs.aws.amazon.com/apigateway/>.
- [2] Amazon Cognito. Amazon cognito user pools. Documentation. Accessed 2026-07-15. URL: <https://docs.aws.amazon.com/cognito/latest/developerguide/cognito-user-pools.html>.
- [3] Amazon DynamoDB. Amazon dynamodb documentation. Documentation. Accessed 2026-07-15. URL: <https://docs.aws.amazon.com/dynamodb/>.
- [4] Amazon Elastic Container Service. RunTask - Amazon Elastic Container Service API Reference. Documentation. Accessed 2026-07-15. URL: https://docs.aws.amazon.com/AmazonECS/latest/APIReference/API_RunTask.html.
- [5] Amazon Elastic Kubernetes Service. Amazon eks documentation. Documentation. Accessed 2026-07-15. URL: <https://docs.aws.amazon.com/eks/>.
- [6] Amazon RDS. Using amazon rds proxy. Documentation. Accessed 2026-07-15. URL: <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/rds-proxy.html>.
- [7] Amazon Web Services. SaaS Lens - AWS Well-Architected Framework. Documentation. Accessed 2026-07-15. URL: <https://docs.aws.amazon.com/wellarchitected/latest/saas-lens/welcome.html>.
- [8] AWS Fargate. Aws fargate documentation. Documentation. Accessed 2026-07-15. URL: https://docs.aws.amazon.com/AmazonECS/latest/developerguide/AWS_Fargate.html.
- [9] AWS Lambda. Aws lambda documentation. Documentation. Accessed 2026-07-15. URL: <https://docs.aws.amazon.com/lambda/>.
- [10] AWS Lambda Powertools. Powertools for aws lambda (python). Documentation. Accessed 2026-07-15. URL: <https://docs.aws.amazon.com/powertools/python/latest/>.
- [11] AWS Secrets Manager. Aws secrets manager documentation. Documentation. Accessed 2026-07-15. URL: <https://docs.aws.amazon.com/secretsmanager/>.
- [12] Cedar Policy Language Authors. Cedar policy language reference guide. Website. Accessed 2026-07-15. URL: <https://docs.cedarpolicy.com/>.

- [13] Roy T. Fielding, Mark Nottingham, and Julian Reschke. RFC 9110: HTTP semantics. RFC Editor, 2022. Accessed 2026-07-15. URL: <https://www.rfc-editor.org/rfc/rfc9110.html>.
- [14] D. Hardt. RFC 6749: The OAuth 2.0 authorization framework. RFC Editor, 2012. Accessed 2026-07-15. URL: <https://www.rfc-editor.org/rfc/rfc6749>.
- [15] Norman Hardy. The confused deputy (or why capabilities might have been invented). In *ACM SIGOPS Operating Systems Review*, volume 22, pages 36–38, 1988.
- [16] Vincent C. Hu, David Ferraiolo, Rick Kuhn, Adam Schnitzer, Kenneth Sandlin, Robert Miller, and Karen Scarfone. NIST SP 800-162: Guide to Attribute Based Access Control (ABAC) Definition and Considerations. National Institute of Standards and Technology, 2014. URL: <https://doi.org/10.6028/NIST.SP.800-162>.
- [17] Istio Authors. Authorization policy. Documentation. Accessed 2026-07-15. URL: <https://istio.io/latest/docs/reference/config/security/authorization-policy/>.
- [18] Linkerd Authors. Authorization policy. Documentation. Accessed 2026-07-15. URL: <https://linkerd.io/2/features/authorization-policy/>.
- [19] Open Policy Agent Contributors. Open policy agent documentation. Website. Accessed 2026-07-15. URL: <https://www.openpolicyagent.org/docs/>.
- [20] OpenID Foundation. OpenID Connect Core 1.0 incorporating errata set 2. Specification. Accessed 2026-07-15. URL: https://openid.net/specs/openid-connect-core-1_0.html.
- [21] Ruoming Pang, Manuel Caceres, Mike Burrows, Zhifeng Chen, Pratik Dave, Nathan Germer, Alexander Golynski, Kevin Graney, Nina Kang, Lea Kissner, et al. Zanzibar: Google’s consistent, global authorization system. In *Proceedings of the USENIX Annual Technical Conference*, 2019. URL: <https://www.usenix.org/conference/atc19/presentation/pang>.
- [22] PostgreSQL Global Development Group. SET - PostgreSQL Documentation. Documentation. Accessed 2026-07-15. URL: <https://www.postgresql.org/docs/current/sql-set.html>.

ABOUT THE AUTHORS

Shayan Ghasemnezhad is Infrastructure and DevOps Lead at Causify AI, Turin, Italy. His research interests include platform engineering and cloud solutions architecture, production AI/ML systems, and secure, reliable infrastructure operations. Ghasemnezhad received his Bachelor’s degree in Digital Management from Ca’ Foscari University of Venice. Contact him at shayan@causify.ai.

Giacinto Paolo (GP) Saggese is Co-founder and CTO at Causify AI, Potomac, Maryland, USA. His research interests include machine learning, fintech, and the commercialization of research. Saggese received his Ph.D. degree in Electrical and Computer Engineering from the University of Illinois Urbana-Champaign. He is an Adjunct Professor at the University of Maryland, College Park. Contact him at gp@causify.ai.

Paul Smith is Co-founder and Chief Scientist at Causify AI, Cary, North Carolina, USA. His research interests include causal inference, Bayesian modeling, and time-series prediction. Smith received his Ph.D. degree in Mathematics from UCLA. Contact him at paul@causify.ai.

Heanh Sok is Software Engineer at Causify AI, College Park, Maryland, USA. His research interests include big data systems, distributed systems, and AI/ML engineering and operations. Sok received his Master’s degree in Data Science from the University of Maryland, College Park. Contact him at h.sok@causify.ai.

Vedanshubhai Joshi is Software Engineer at Causify AI, Indiana, USA. His research interests include platform infrastructure, multi-tenant API design, and data engineering pipelines. Joshi received his Master’s degree in Computer Science from Purdue University. Contact him at v.joshi@causify.ai.